

PERTEMUAN 2

OBJEK PRIMITIF, ALGORITMA GARIS DAN LINGKARAN

Kemampuan Akhir yang Diharapkan (KAD):

Mahasiswa mampu mengimplementasikan algoritma dasar untuk menggambar bentuk 2D

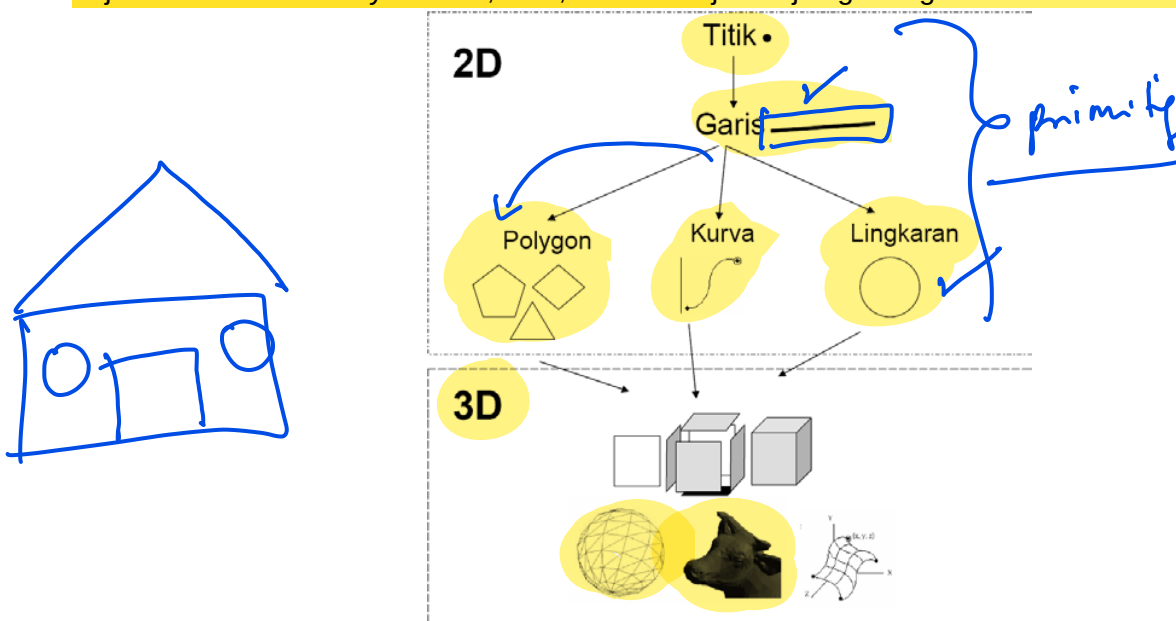
Pokok Bahasan:

1. Pengertian objek primitif
2. Penggambaran titik dan garis
3. Algoritma pembentukan garis (Brute Force, DDA dan Bresenham)
4. Implementasi Penggambaran Garis dengan Java
5. Pengertian objek lingkaran
6. Simetris delapan titik
7. Algoritma Midpoint Lingkaran
8. Implementasi Penggambaran Lingkaran dengan Java
9. Soal Latihan

2.1 Pengertian objek primitif

Gambar dapat dijelaskan dengan beberapa cara, bila menggunakan raster display, gambar ditentukan oleh satu set intensitas untuk posisi display pada display. Sedangkan dengan scene tampilan gambar dengan loading array dari pixel ke dalam buffer atau dengan mengkonversikan scan dari grafik geometri tertentu ke dalam pola pixel. Paket grafika dilengkapi dengan fungsi untuk menyatakan scene dalam bentuk struktur. Paket pemrograman grafika dilengkapi dengan fungsi untuk menyatakan scene dalam bentuk struktur dasar geometri yang disebut output primitif, dengan memasukkan output primitif tersebut sebagai struktur yang lebih kompleks.

Gambar 2.1 menunjukkan bahwa dari sebuah titik dapat dibentuk objek garis, dimana garis dibentuk dari 2 titik. Dari garis dapat dibentuk poligon, kurva maupun lingkaran. Dengan dasar bangun ini maka dapat dibentuk objek-objek lain yang lebih kompleks di antara objek-objek 3 dimensi misalnya kubus, bola, bahkan objek-objek gabungan semua elemen.

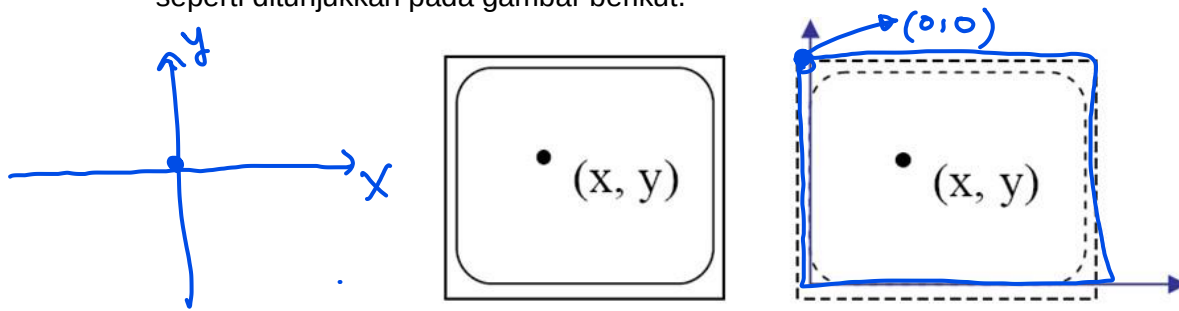


Gambar 2.1 Elemen-elemen Pembentuk Objek Grafis

2.2 Penggambaran Titik dan Garis

Pembentukan titik dilakukan dengan mengkonversi suatu posisi titik koordinat dengan program aplikasi ke dalam suatu operasi tertentu menggunakan output. Random-scan (vektor) system menyimpan instruksi pembentukan titik pada display list dan nilai koordinat menentukan posisi pancaran electron ke arah lapisan fosfor pada layer. Garis dibuat dengan menentukan posisi titik diantara titik awal dan akhir dari suatu garis.

Perlu diperhatikan bahwa sistem koordinat biasa dan sistem koordinat grafika sedikit berbeda seperti ditunjukkan pada gambar berikut.



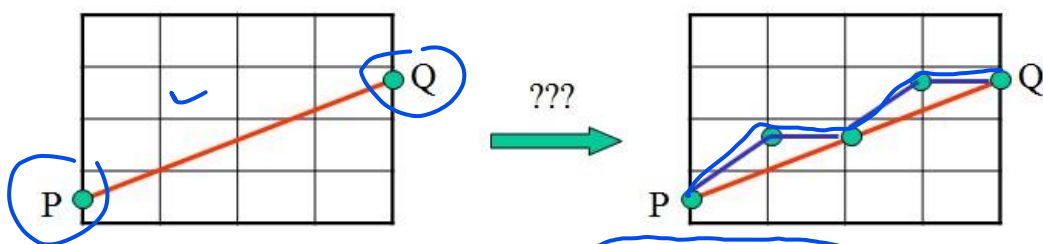
Perhatikan bahwa sistem koordinat dalam pemrograman grafis berbeda dengan model koordinat kartesius

Gambar 2.2 Sistem Koordinat pada Layar dan Kartesius

Pada sistem grafika, posisi (0,0) ada pada kiri atas dari layar, sedangkan menurut sistem koordinat kartesius posisi (0,0) ada di tengah-tengah bidang gambar.

2.3 Algoritma Pembentukan Garis

Pada dasarnya, algoritma penggambaran atau pembentukan garis berusaha mencari suatu cara membentuk garis sedemikian rupa sehingga masalah jaggies dapat dihindarkan se optimal mungkin. Gambar di bawah ini mengilustrasikan keadaan tersebut.



Gambar 2.3 Pembentukan Garis Secara Diskrit

A. Algoritma Brute Force

Persamaan garis menurut koordinat Cartesian adalah:

$$y = m \cdot x + b$$

** Fungsi Linear*

dimana m adalah slope (kemiringan) dari garis yang dibentuk oleh dua titik yaitu (x_1, y_1) dan (x_2, y_2) . Untuk penambahan x sepanjang garis yaitu dx akan mendapatkan penambahan y sebesar $\Delta y = m \cdot \Delta x$. Dari formulasi di atas dikembangkan algoritma dasar untuk penggambaran garis.

Contoh:

Diketahui dua buah titik **A(2, 1)** dan **B(6, 4)**. Tentukan titik-titik digital yang dilalui oleh garis yang melalui kedua titik tersebut!

Jawab:

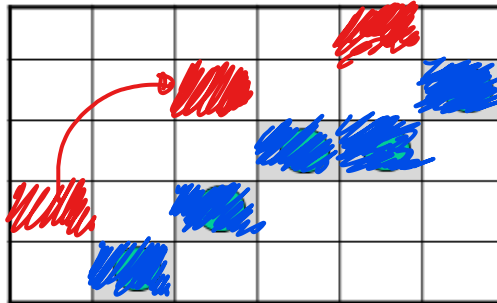
Hitung nilai m

$$m = \frac{y_2 - y_1}{x_2 - x_1} = \frac{4 - 1}{6 - 2} = \frac{3}{4} = 0.75$$

Kemudian buat tabel berikut:

x	Δx	x^*	y	Δy	y^*	[x]	[y]
2	1	2	1	0	1	2	1
3	1	3	1.75	0.75	1.75	3	2
4	1	4	2.50	0.75	2.50	4	3
5	1	5	3.25	0.75	3.25	5	3
6	1	6	4.00	0.75	4.00	6	4

Jadi titik-titik digitalnya adalah (2,1), (3,2), (4,3), (5,3) dan (6,4).



Pseudocode Algoritma Penggambaran Garis Brute Force

```

method lineDraw1(x1,y1,x2,y2) {
    dy = y2-y1;
    dx = x2-x1;
    m = dy/dx;
    c = y1-(m*x1);
    for(int x=x1; x<x2; x++) {
        y = (m*x)+c;
        plotpoint(x,y)
    }
} //lineDraw

```

Garis garis solid

$x = x + 1$

dot line

B. Algoritma DDA (Digital Differential Analyzer)

DDA adalah algoritma pembentukan garis berdasarkan perhitungan Δx dan Δy , menggunakan rumus $y = m \cdot \Delta x$. Garis dibuat dengan menentukan dua endpoint yaitu titik awal dan titik akhir. Setiap koordinat titik yang membentuk garis diperoleh dari perhitungan, kemudian

dikonversikan menjadi nilai integer. Langkah-langkah pembentukan menurut algoritma DDA, yaitu:

1. Tentukan dua titik yang akan dihubungkan.
2. Tentukan salah satu titik sebagai titik awal (x_0, y_0) dan titik akhir (x_1, y_1).
3. Hitung $\Delta x = x_1 - x_0$ dan $\Delta y = y_1 - y_0$.
4. Tentukan *step*, yaitu jarak maksimum jumlah penambahan nilai x maupun nilai y dengan cara :
 - bila nilai $|\Delta y| > |\Delta x|$ maka $step = \text{nilai } |\Delta y|$.
 - bila tidak maka $step = |\Delta x|$.
5. Hitung penambahan koordinat *pixel* yaitu $x_increment = \Delta x / step$ dan $y_increment = \Delta y / step$.
6. Koordinat selanjutnya ($x + x_increment, y + y_increment$).
7. Posisi *pixel* pada *layer* ditentukan dengan pembulatan nilai koordinasi tersebut.
8. Ulangi step 6 dan 7 untuk menentukan posisi *pixel* selanjutnya, sampai $x = x_1$ dan $y = y_1$.

Contoh :

Untuk menggambarkan algoritma DDA dalam pembentukan suatu garis yang menghubungkan titik (10,10) dan (17,16)!

Jawab:

Pertama tentukan dx dan dy , kemudian dicari $step$ untuk mendapatkan $x_increment$ dan $y_increment$.

$$\Delta x = x_1 - x_0 = 17 - 10 = 7$$

$$\Delta y = y_1 - y_0 = 16 - 10 = 6$$

selanjutnya hitung dan bandingkan nilai absolutnya.

$$|\Delta x| = 7 \quad ? \text{ step?}$$

$$|\Delta y| = 6$$

karena $|\Delta x| > |\Delta y|$, maka $step = |\Delta x| = 7$, maka diperoleh:

$$x_increment = 7/7 = 1$$

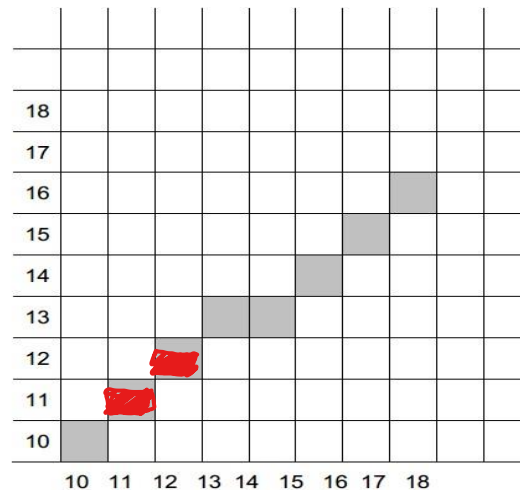
$$y_increment = 6/7 = 0.86$$

k	x	y	Round(x)	Round(y)
-	10	10	10	10
0	11	10.86	11	11
1	12	11.72	12	12
2	13	12.58	13	13
3	14	13.44	14	13
4	15	14.3	15	14
5	16	15.16	16	14
6	17	16.02	17	16

float

Pembulatan integer

x, y



Pseudocode Algoritma *Digital Differential Analyzer (DDA)*

```

Int dx = x2-x1;
Int dy = y2-y1;
Int steps, k, x1, y1, x2, y2;
Float x_inc, y_inc;
Float x = x1;
Float y = y1;
If (abs(dx) > abs(dy))
    steps = abs(dx);
else
    steps = abs(dy);
X_inc = dx / (float)steps;
Y_inc = dy / (float)steps;
setPixel(Round(x), Round(y));
For (k=0; k<steps; k++) {
    x+=x_inc;
    y+=y_inc;
    setPixel(Round(x), Round(y));
}

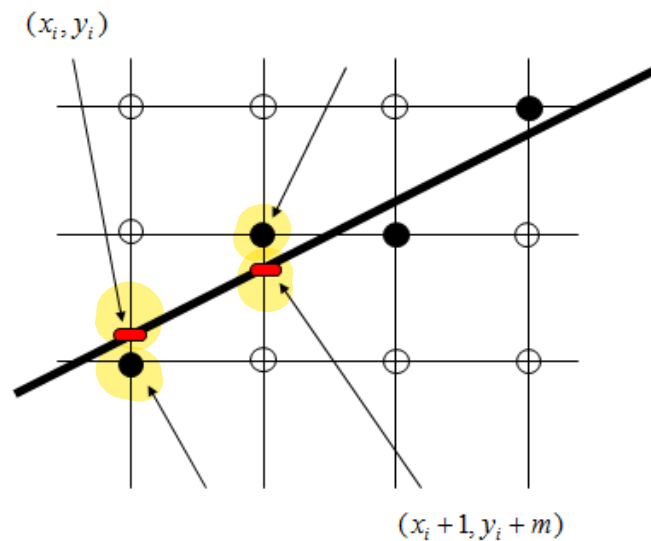
```

Algoritma-algoritma sebelumnya secara umum sudah dapat menjawab masalah penggambaran garis, namun dalam komputasinya masih mengandung perhitungan aritmatik yang secara analisis membuat kebutuhan sumber daya komputasi menjadi besar karena operasi *floating points*. Untuk itu dikembangkan algoritma lain yaitu **Algoritma Bresenham** yang bekerja pada domain bilangan *integer* sehingga walaupun algoritma menjadi panjang namun lebih efisien.

C. Algoritma Bresenham

Prosedur untuk menggambar kembali garis dengan membulatkan nilai x atau y ke bilangan *integer* membutuhkan waktu, serta *variable* (x, y) dan m merupakan bilangan *real* karena kemiringan merupakan nilai pecahan. Bresenham mengembangkan algoritma klasik yang lebih menarik, karena hanya menggunakan perhitungan matematika dengan bilangan *integer*.

Dengan demikian tidak perlu membulatkan nilai posisi setiap *pixel* setiap waktu. Algoritma garis Bresenham disebut juga **midpoint line algorithm** adalah algoritma konversi penambahan nilai *integer* yang juga dapat diadaptasi untuk menggambar sebuah lingkaran.



Gambar 2.4 Penggambaran Garis dengan Teknik Bresenham

Langkah-langkah untuk membentuk garis menurut algoritma ini adalah :

1. Tentukan dua titik yang akan dihubungkan dalam pembentukan garis.
2. Tentukan salah satu titik disebelah kiri sebagai titik awal (x_0, y_0) dan titik lainnya sebagai titik akhir (x_1, y_1) .
3. Hitung Δx , Δy , $2\Delta y$, dan $2\Delta y - 2\Delta x$.
4. Hitung parameter $p_0 = 2\Delta y - \Delta x$.
5. Untuk setiap x_k sepanjang jalur garis, dimulai dengan $k = 0$
 bila $p_k < 0$ maka titik selanjutnya:
 (x_{k+1}, y_k) dan $p_{k+1} = p_k + 2\Delta y$
 bila tidak maka titik selanjutnya
 (x_{k+1}, y_{k+1}) dan $p_{k+1} = p_k + (2\Delta y - 2\Delta x)$.
6. Ulangi langkah nomor 5 untuk menentukan posisi *pixel* selanjutnya, sampai $x = x_1$ atau $y = y_1$.

Contoh:

Untuk menggambarkan algoritma Bresenham dalam pembentukan suatu garis yang menghubungkan titik $(10,10)$ dan $(17,16)$!

Jawab:

Pertama tentukan bahwa titik $(10,10)$ berada disebelah kiri merupakan titik awal, sedangkan $(17,16)$ merupakan titik akhir. Posisi yang membentuk garis dapat ditentukan dengan perhitungan sebagai berikut:

$$\Delta x = x_1 - x_0$$

$$\Delta y = y_1 - y_0$$

$$\Delta x = 17 - 10 = 7$$

$$\Delta y = 16 - 10 = 6$$

$$\text{parameter } p_0 = 2\Delta y - \Delta x$$

$$p_0 = 2(6) - 7$$

$$= 5$$

$$p_0 = (2 \cdot 6) - 7 = 5$$

$$\text{increment } 2\Delta y = 12$$

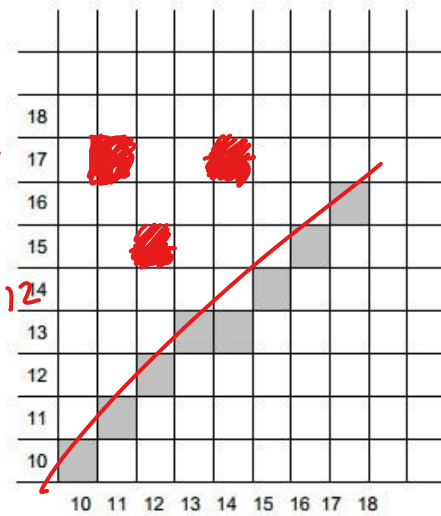
$$2\Delta y - 2\Delta x = 12 - 14 = -2$$

k	p_k	(x_{k+1}, y_{k+1})
$p_0 \rightarrow 5 > 0$		(10, 10)
$p_1 \rightarrow 3 > 0$		(11, 11)
$p_2 \rightarrow 1 > 0$		(12, 12)
$p_3 \rightarrow -1 < 0$		(13, 13)
$p_4 \rightarrow 11 > 0$		(14, 13)
$p_5 \rightarrow 9 > 0$		(15, 14)
$p_6 \rightarrow 7 > 0$		(16, 15)
		(17, 16)

 x_1, y_1

$$p_k < 0 \rightarrow (x+1, y) \rightarrow p_{k+1} = p_k + 12$$

$$p_k > 0 \rightarrow (x+1, y+1) \rightarrow p_{k+1} = p_k - 2$$



Pseudocode Algoritma Bresenham untuk Penggambaran Garis

```

Procedure BresenhamLine(x1, y1, x2, y2)
  dx ← abs(x2 - x1)
  dy ← abs(y2 - y1)
  sx ← 1 if x1 < x2 else -1
  sy ← 1 if y1 < y2 else -1
  err ← dx - dy

  x ← x1
  y ← y1

  While true do
    SetPixel(x, y)    // plot the current point

    If x = x2 and y = y2 then
      Exit loop      // line finished

    e2 ← 2 * err

    If e2 > -dy then
      err ← err - dy
      x ← x + sx

    If e2 < dx then
      err ← err + dx
      y ← y + sy
  EndProcedure
  
```

2.4 Implementasi Algoritma Penggambaran Garis dengan Java

A. Algoritma Brute Force

Kode lengkap untuk penggambaran garis menggunakan Algoritma Brute Force:

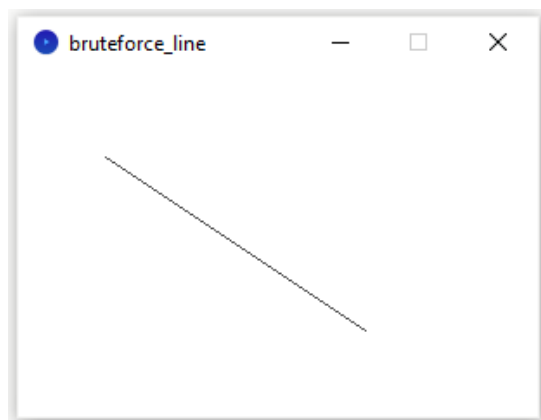
```
int x1 = 50, y1 = 50;
int x2 = 200, y2 = 150;

void setup() {
    size(300, 200);
    background(255);
    stroke(0);

    drawBruteForceLine(x1, y1, x2, y2);
}

void drawBruteForceLine(int x1, int y1, int x2, int y2) {
    float dx = x2 - x1;
    float dy = y2 - y1;

    // Avoid division by zero
    if (abs(dx) >= abs(dy)) {
        // Line is more horizontal
        float m = dy / dx;
        for (int x = x1; x <= x2; x++) {
            float y = y1 + m * (x - x1);
            point(x, round(y));
        }
    } else {
        // Line is more vertical
        float m_inv = dx / dy;
        for (int y = y1; y <= y2; y++) {
            float x = x1 + m_inv * (y - y1);
            point(round(x), y);
        }
    }
}
```



B. Algoritma DDA

Kode lengkap untuk penggambaran garis menggunakan Algoritma DDA:

```
int x1 = 50, y1 = 50;
int x2 = 200, y2 = 150;

void setup() {
    size(300, 200);
    background(255);
    stroke(0);

    drawDDALine(x1, y1, x2, y2);
}

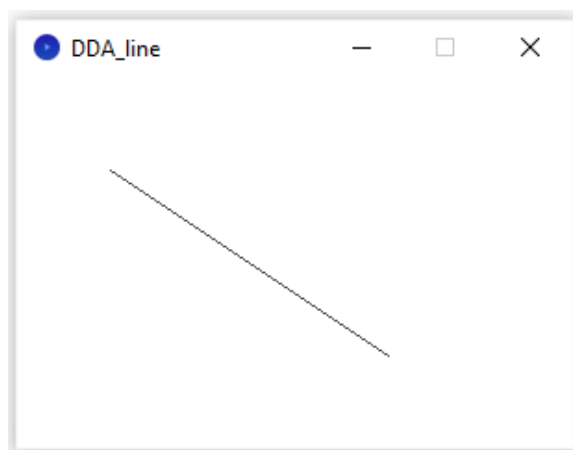
void drawDDALine(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1;
    int dy = y2 - y1;

    int steps; // choose the maximum of dx and dy
    if (abs(dx) >= abs(dy)) {
        steps = abs(dx);
    }else{
        steps = abs(dy);
    }

    float xInc = dx / (float) steps;
    float yInc = dy / (float) steps;

    float x = x1;
    float y = y1;

    for (int k = 0; k <= steps; k++) {
        point(round(x), round(y)); // round to nearest integer to
        plot pixel
        x += xInc;
        y += yInc;
    }
}
```



C. Algoritma Bresenham

Kode lengkap untuk penggambaran garis menggunakan Algoritma Bresenham:

```
int x1 = 50, y1 = 50;
int x2 = 200, y2 = 150;

void setup() {
  size(300, 200);
  background(255);
  stroke(0);

  drawBresenhamLine(x1, y1, x2, y2);
}

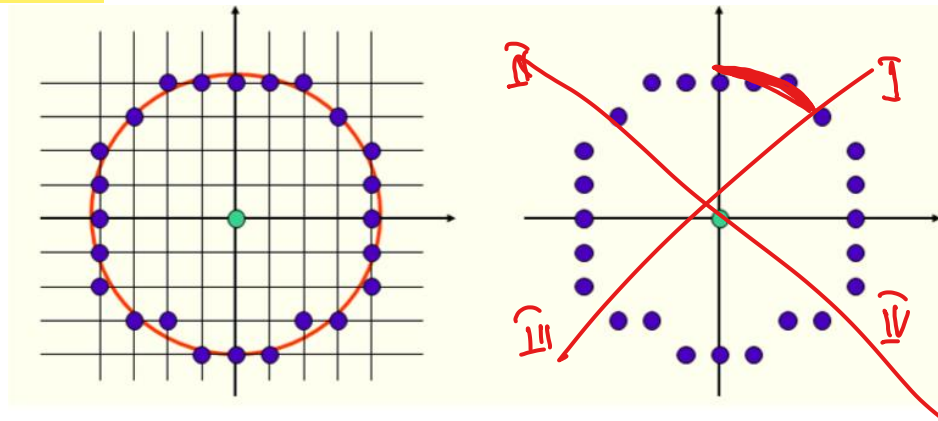
void drawBresenhamLine(int x1, int y1, int x2, int y2) {
  int dx = abs(x2 - x1);
  int dy = abs(y2 - y1);
  int sx = x1 < x2 ? 1 : -1;
  int sy = y1 < y2 ? 1 : -1;
  int err = dx - dy;
  int x = x1;
  int y = y1;
  while (true) {
    point(x, y);
    if (x == x2 && y == y2) break;
    int e2 = 2 * err;
    if (e2 > -dy) {
      err -= dy;
      x += sx;
    }
    if (e2 < dx) {
      err += dx;
      y += sy;
    }
  }
}
```



2.5 Pengertian objek lingkaran

Lingkaran adalah kumpulan dari titik-titik yang memiliki jarak dari titik pusat yang sama untuk semua titik. Lingkaran dibuat dengan menggambarkan seperempat lingkaran, karena bagian lain dapat dibuat sebagai bagian yang simetris. Penambahan x dapat dilakukan dari 0 ke r sebesar unit *step*, yaitu menambahkan $\pm y$ untuk setiap *step*.

Pada dasarnya, mirip dengan penggambaran garis, algoritma penggambaran lingkaran pun berusaha mencari solusi optimal sedemikian rupa sehingga lingkaran yang dibentuk adalah sesempurna mungkin. Gambar di bawah ini memberikan ilustrasi penggambaran lingkaran.



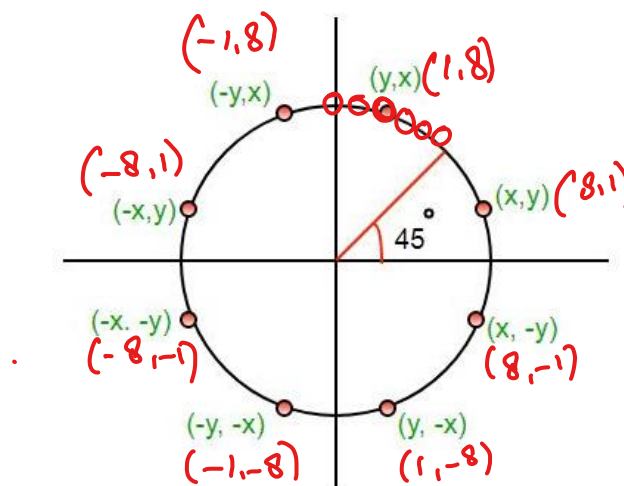
Gambar 2.5 Ilustrasi Penggambaran Lingkaran

2.6 Simetris delapan titik

Pada umumnya, lingkaran digunakan sebagai komponen dari suatu gambar. Prosedur untuk menampilkan lingkaran dan elips dibuat dengan persamaan dasar dari lingkaran $x^2 + y^2 = r^2$. Proses pembuatan lingkaran dapat dilakukan dengan menentukan satu titik awal. Bila titik awal pada lingkaran (x,y) , maka terdapat tiga posisi lain, sehingga dapat diperoleh delapan titik. Dengan demikian, hanya diperlukan untuk menghitung *segmen* 45° dalam menentukan lingkaran selengkapnya. Delapan titik simetris tersebut, yaitu:

- Kuadran I $(x,y),(y,x)$
- Kuadran II $(-x,y),(-y,x)$
- Kuadran III $(-x,-y),(-y,-x)$
- Kuadran IV $(x,-y),(y,-x)$

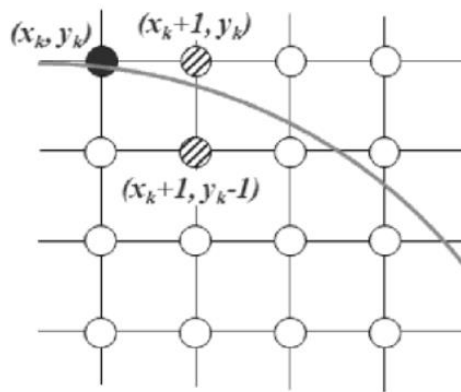
Gambar di bawah ini mengilustrasikan kuadran tersebut.



Gambar 2.6 Delapan titik simetris pada lingkaran

2.7 Algoritma midpoint lingkaran

Algoritma lingkaran midpoint disebut juga algoritma lingkaran Bresenham. Algoritma yang digunakan membentuk semua titik berdasarkan titik pusat dengan penambahan semua jalur disekeliling lingkaran. Komputasi untuk membuat kurva lingkaran dimulai dengan mengidentifikasi bagian-bagian dari lingkaran yang dapat ditentukan dengan menggunakan sifat simetri, hal ini dilakukan dengan cara membagi lingkaran dengan masing-masing mempunyai sudut sebesar 45° , sehingga dalam sebuah lingkaran dibagi menjadi 8 (delapan) bagian. Dalam hal ini hanya diperhatikan bagian 45° dari suatu lingkaran, yaitu oktan kedua dari $x = 0$ ke $x = R/\sqrt{2}$, dan menggunakan prosedur *circle point* untuk menampilkan titik dari seluruh lingkaran. Perhatikan berikut:



Gambar 2.7 Garis lingkaran berada di antara titik $(x_k + 1, y_k)$ dan $(x_k + 1, y_k - 1)$. Angap bahwa kita mempunyai titik (x_k, y_k) . Selanjutnya adalah memilih titik $(x_k + 1, y_k)$ atau $(x_k + 1, y_k - 1)$ yang paling mendekati lingkaran. Persamaan untuk lingkaran adalah:

$$f(x, y) = x^2 + y^2 - r^2$$

sehingga:

$$f_{\text{circle}}(x, y) \begin{cases} < 0, \text{ bila } (x, y) \text{ di dalam garis lingkaran} \\ = 0, \text{ bila } (x, y) \text{ di garis lingkaran} \\ > 0, \text{ bila } (x, y) \text{ di luar garis lingkaran} \end{cases}$$

fungsi lingkaran menggambarkan posisi *midpoint* antara *pixel* yang terdekat dengan jalur lingkaran setiap *step*. Fungsi lingkaran menentukan *parameter* pada algoritma lingkaran. Untuk memilih titik $(x_k + 1, y_k)$ atau $(x_k + 1, y_k - 1)$ yang paling mendekati lingkaran, kriteria yang digunakan adalah titik tengah atau $(x_k + 1, y_k - \frac{1}{2})$, yaitu:

$$p_k = f(x_k + 1, y_k - \frac{1}{2}) = (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2$$

jika $p_k < 0$ titik tengah berada di dalam garis lingkaran, maka titik $(x_k + 1, y_k)$ dipilih.

jika $p_k > 0$ titik tengah berada di luar garis lingkaran, maka titik $(x_k + 1, y_k - 1)$ dipilih.

Hitung:

$$p_{k+1} = f(x_{k+1} + 1, y_{k+1} - \frac{1}{2}) = (x_{k+1} + 1)^2 + (y_{k+1} - \frac{1}{2})^2 - r^2$$

atau:

$$p_{k+1} = p_k + 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1$$

dimana y_{k+1} adalah y_k atau $y_k - 1$ tergantung tanda dari p_k . Dalam hal ini kriteria pertama yang dihitung adalah:

$$p_0 = f(1, r - \frac{1}{2}) = 1 + (r - \frac{1}{2})^2 - r^2 = 5/4 - r$$

jika jari-jari lingkaran bernilai *integer* (bilangan bulat), kita bisa membulatkan nilai p_0 menjadi berikut:

$$p_0 = 1 - r$$

kemudian jika $p_k < 0$, maka:

$$p_{k+1} = p_k + 2x_{k+1} + 1$$

jika $p_k > 0$, maka:

$$p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_k + 1$$

Dimana $2x_{k+1} = 2x_k + 2$ dan $2y_{k+1} = 2y_k - 2$

Dari sini diperoleh langkah-langkah algoritma pembentuk lingkaran.

Langkah-langkah algoritma lingkaran *midpoint* adalah:

1. Tentukan radius (jari-jari) r dan titik pusat lingkaran (x_c, y_c) , kemudian diperoleh $(x_0, y_0) = (0, r)$.
2. Hitung nilai dari parameter
 ~~$p_0 = 5/4 - r$ jika jari-jari r pecahan~~
 $p_0 = 1 - r$ jika jari-jari r *integer* (bilangan bulat)
3. Untuk setiap posisi x_k , dimulai dengan nilai awal $k=0$ berlaku ketentuan sbb:
 - Bila $p_k < 0$, maka titik selanjutnya adalah (x_{k+1}, y_k)
 $p_{k+1} = p_k + 2x_{k+1} + 1$
 - Bila $p_k > 0$, maka titik selanjutnya adalah $(x_k + 1, y_k - 1)$
 $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_k + 1$

Dimana $2x_{k+1} = 2x_k + 2$ dan $2y_{k+1} = 2y_k - 2$
4. Tentukan titik simetris pada ketujuh oktan yang lain.
5. Gerakkan setiap posisi pixel (x, y) pada garis melingkar dari lingkaran dengan titik pusat (x_c, y_c) dan tentukan nilai koordinat :
 $x = x + x_c$ dan $y = y + y_c$
6. Ulangi langkah ke 3 – 5, sampai dengan $x \geq y$.

Contoh:

Buatlah gambar kurva lingkaran dengan pusat lingkaran di koordinat $(4, 6)$ dan jari-jari 8. Perhitungan berdasarkan dari oktan kuadran pertama dimana $x = 0$ sampai $x = y$. koordinat titik awal dimulai dari $(x, r) = (0, 8)$. Karena jari-jari r bulat, maka gunakan $p_0 = 1 - r$.

$$x = x + 4 \quad y = y + 6$$

$$p_0 = 1 - r = 1 - 8 = -7$$

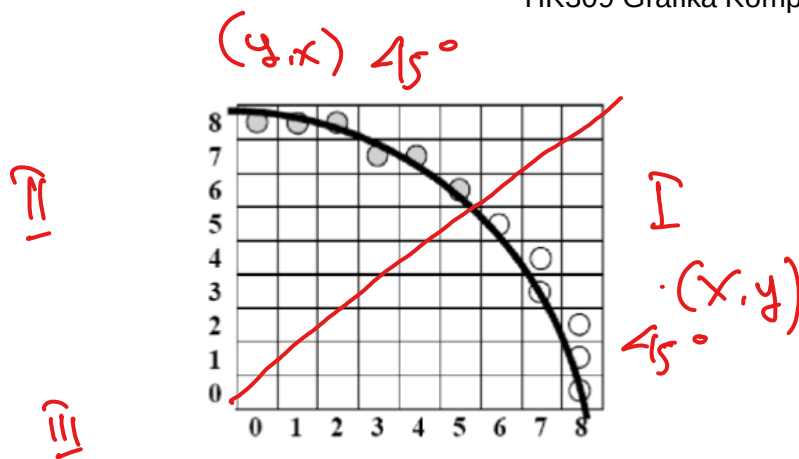
Jawab:

Tabel 2.1 Posisi pixel pada pembentukan lingkaran dengan titik pusat $(0, 0)$ dan radius 8

k	p_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$
-	-	(0, 8)	0	16
$p_0 = -7 < 0$		(1, 8)	2	16
$p_1 = -4 < 0$		(2, 8)	4	16
$p_2 = 1 > 0$		(3, 7)	6	14
$p_3 = -6 < 0$		(4, 7)	8	14
$p_4 = 3 > 0$		(5, 6)	10	12
$p_5 = 2 > 0$		(6, 5)	12	10

$$\begin{aligned}
 x &= x + x_c \\
 y &= y + y_c \\
 4, 14 \\
 5, 14 \\
 6, 14 \\
 7, 13 \\
 8, 13 \\
 9, 12 \\
 10, 9
 \end{aligned}$$

$x \geq y \rightarrow \text{Selesai}$



Gambar 2.8 Posisi pixel pada pembentukan lingkaran dengan titik pusat (0,0) dan radius 8

Pseudocode Algoritma Bresenham untuk Lingkaran

```

BresenhamCircle(Xc, Yc, R)
Set X = 0 and Y = R
Set D = 3 - 2R
Repeat While (X < Y)
    Call DrawCircle(Xc, Yc, X, Y)
    Set X = X + 1
    If (D < 0) Then
        D = D + 4X + 6
    Else
        Set Y = Y - 1
        D = D + 4(X - Y) + 10
    Call Draw Circle(Xc, Yc, X, Y)
DrawCircle(Xc, Yc, X, Y)
Call PutPixel(Xc + X, Yc + Y)
Call PutPixel(Xc - X, Yc + Y)
Call PutPixel(Xc + X, Yc - Y)
Call PutPixel(Xc - X, Yc - Y)
Call PutPixel(Xc + Y, Yc + X)
Call PutPixel(Xc - Y, Yc + X)
Call PutPixel(Xc + Y, Yc - X)
Call PutPixel(Xc - Y, Yc - X)

```

2.8 Implementasi Algoritma Penggambaran Lingkaran dengan Java

Kode lengkap untuk penggambaran lingkaran menggunakan algoritma *midpoint* atau *bresenham*:

```

void setup() {
    size(300, 300);
    background(255);
    stroke(0);

    int centerX = width / 2;
    int centerY = height / 2;
    int radius = 100;
}

```

```

    bresenhamCircle(centerX, centerY, radius);
}

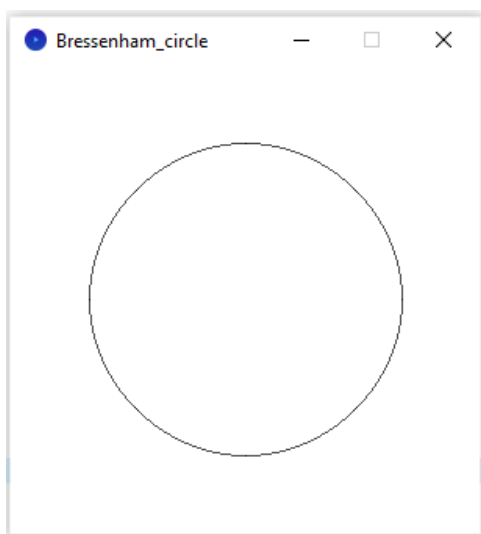
void bresenhamCircle(int xc, int yc, int r) {
    int x = 0;
    int y = r;
    int d = 3 - 2 * r;

    drawCirclePoints(xc, yc, x, y);

    while (x <= y) {
        x = x + 1;
        if (d < 0) {
            d = d + 4 * x + 6;
        } else {
            y = y - 1;
            d = d + 4 * (x - y) + 10;
        }
        drawCirclePoints(xc, yc, x, y);
    }
}

void drawCirclePoints(int xc, int yc, int x, int y) {
    point(xc + x, yc + y);
    point(xc - x, yc + y);
    point(xc + x, yc - y);
    point(xc - x, yc - y);
    point(xc + y, yc + x);
    point(xc - y, yc + x);
    point(xc + y, yc - x);
    point(xc - y, yc - x);
}

```



2.9 Soal Latihan

1. Dengan menggunakan algoritma brute force, tentukan koordinat titik-titik digital untuk garis yang dibentuk oleh dua titik berikut!

- a. (-5,5) dan (1,2)
- b. (4,3) dan (8,-2)
- c. (2,3) dan (5,3)
- d. (2,3) dan (2,5)
- e. (6,4) dan (2,1)

Catatan: Pada saat anda mengerjakan soal di atas, akan ditemui masalah yang tidak dapat diselesaikan oleh algoritma brute force, untuk itu selesaikan dengan algoritma lain yang sudah dijelaskan sebelumnya.

2. Dengan menggunakan algoritma DDA, tentukan koordinat titik-titik digital untuk garis yang dibentuk oleh dua titik sebagai berikut!

- a. (-5,5) dan (1,2)
- b. (4,3) dan (8,-2)
- c. (2,3) dan (5,3)
- d. (2,3) dan (2,5)
- e. (6,4) dan (2,1)

Catatan: Apakah semua soal dapat diselesaikan?

3. Buatlah lingkaran dengan titik pusat (0,0) dan radiusnya 9 dengan algoritma Midpoint (Bresenham)!

Daftar Pustaka

1. Andono, Pulung Nurtantio., Sutojo, T. 2016. Konsep Grafika Komputer. Yogyakarta: CV Andi Offset.
2. Klawonn, Frank. 2012. *Introduction to Computer Graphics Using Java 2D and 3D 2nd Edition*. Springer-Verlag: London.